

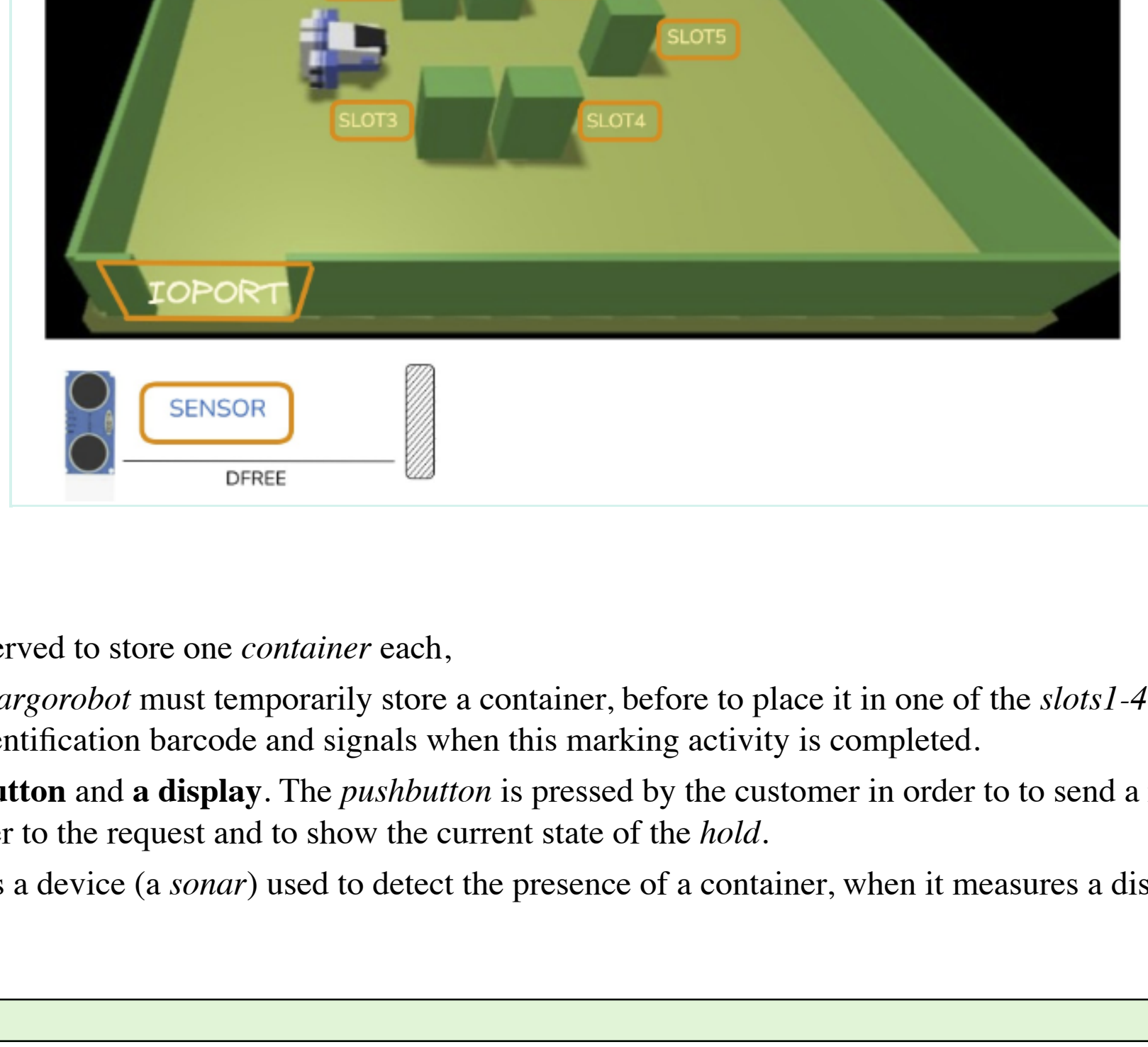
TF26 - Sprint0

Goal Sprint0: Analisi dei Requisiti

Introduzione

A *Maritime Cargo shipping company* (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**).

The *hold* is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 `slots` to store the containers and a slot named **slot5**.



In the picture above:

- The **slots1-4** depict the hold areas reserved to store one *container* each,
- The **slot5** depicts an area, where the *cargorobot* must temporarily store a container, before to place it in one of the *slots1-4*. During temporary storage, a "marker" device labels the container with an identification barcode and signals when this marking activity is completed.
- The **IOPort** is a device with a **pushbutton** and a **display**. The *pushbutton* is pressed by the customer in order to send a request to load a container on the cargo. The *display* is used to show the answer to the request and to show the current state of the *hold*.
- The **sensor** associated to the *IOPort* is a device (a *sonar*) used to detect the presence of a container, when it measures a distance D, such that $D < DFREE/2$, during a reasonable time (e.g. 3 secs).

Requirements

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer *retrylater*, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the `slots1-4` are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request* to load is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message **"Service working"**, when it is all is going well
- the message **"Out of service"** if the *sonar sensor* measures a distance $D > DFREE$ for at least 3 secs (perhaps a failure of the *sonar*).

Requirement analysis

Il sistema software preso in esame è composto dalle seguenti entità:

Cargorobot (DDR)

È un'entità software attiva che rappresenta il comportamento del robot fisico a guida differenziale (DDR) operante all'interno della stiva ed è modellata come [attore](#). Riceve ed esegue i comandi di macro-movimentazione inviati dall'orchestratore centrale per spostarsi sequenzialmente tra l'IOPort, lo slot 5 e gli alloggiamenti finali (slot 1-4).

Il committente ha messo a disposizione più varianti di **VirtualRobot26**: **RobotSmart26**, **RobotService26**, **RobotObj26**. Si valuta di utilizzare RobotSmart26 in quanto come servizio permette di colmare maggiormente l'abstraction gap e fornisce un ambiente virtuale in cui visualizzare i movimenti del robot (che coincide con quanto richiesto nei requisiti). Questo modulo solleva il nostro software dall'onere di calcolare i comandi di movimento di basso livello (avanza, ruota, stop), poiché riceve in input le coordinate cartesiane di destinazione e calcola autonomamente il percorso ottimale sulla griglia.

IOPort

Rappresenta il punto di interazione fisico/logico posizionato all'ingresso della stiva, strutturato come un nodo indipendente che espone una Web GUI (come indicato dal committente). Gestisce i flussi di input e output dell'utente disaccoppiandoli dalla logica aziendale core ed è modellato come un [attore](#).

- Il suo componente di input (pushbutton) è un elemento reattivo che cattura l'interazione del cliente (pressione del pulsante) e la traduce in una richiesta formale di carico (button_pushed) indirizzata verso il sistema.
- Il suo componente di output (display) si comporta come un terminale passivo incaricato di mostrare visivamente le informazioni trasmesse dal servizio. Mostra in tempo reale lo stato di occupazione dei quattro slot e lo stato del sistema: "Service working" o "Out of service".

Vengono formalizzati quindi i [messaggi](#):

Request loadrequest	: loadrequest(IOPORT_STATE)
Reply retrylater	: retrylater(CAUSE_HOLD_STATE) for loadrequest
Reply reject	: reject(HOLD_STATE) for loadrequest
Reply engaged	: engaged(RESERVED_SLOT_HOLD_STATE) for loadrequest

Sensore / Sonar

Si tratta di un'entità software proattiva ([attore](#)) che rappresenta un dispositivo fisico di telemetria posizionato strategicamente in corrispondenza dell'area di accesso dell'IOPort. Effettua un campionamento continuo dello spazio antistante, calcolando la distanza D dall'ostacolo più vicino. Questo software di basso livello trasmette i dati grezzi sottoforma di stringhe formattate come distance(DIRECTION, D) per consentire la verifica dei vincoli temporali e di soglia del sistema:

- se una misurazione $D < DFREE/2$ persiste per un intervallo temporale ragionevole (es. almeno 3 secondi), viene validata la presenza fisica di un carico da elaborare;
- se una misurazione $D > DFREE$ persiste per più di 3 secondi, viene notificata una condizione di potenziale guasto hardware (stato Out of service).

Dopo aver parlato con il committente si è stabilito che il sonar è un dispositivo fisico collegato ad un Pico W. Ciò rende il sistema distribuito ed eterogeneo.

Dispositivo Marker (Slot 5)

Si tratta di un dispositivo di scansione e identificazione localizzato in corrispondenza dello slot 5, modellato come [attore](#), l'area riservata alla sosta temporanea.

Si attiva non appena viene segnalata la deposizione di un container nello slot 5, avviando un'operazione automatica di etichettatura per applicare un codice a barre identificativo sul carico. Una volta completata l'operazione, genera un messaggio di notifica.

Stiva (Hold)

La mappa fornita da RobotSmart26 ha le informazioni legate alla topologia degli slot, ma non ha nessuna nozione dello stato. Perciò sarà necessario un componente software passivo modellabile come un oggetto Java puro (**POJO**) che tiene traccia in memoria dello stato della stiva. Modella e traccia la situazione dei quattro slot di deposito principali (slot 1-4) e dello slot 5 temporaneo. Ciascun alloggiamento viene catalogato attraverso tre stati logici ben definiti: *free* (disponibile), *reserved* (assegnato a una richiesta accettata, ma non ancora occupato fisicamente dal container) e *occupied* (container depositato). Lo stato viene formalizzato come segue:

public enum SlotState { FREE, RESERVED, OCCUPIED }
--

L'entità incapsula la logica algoritmica per determinare se la stiva è saturata (isFull()) e individua e riserva il primo slot disponibile in ordine sequenziale non appena viene convalidata una nuova richiesta di accesso.

Come la topologia fornita da RobotSmar26 e lo stato degli slot saranno messi in relazione tra loro verrà analizzato in una fase successiva.

Led

Rappresenta un'entità software passiva di output associata a un attuatore luminoso fisico. La sua responsabilità è quella di fornire una segnalazione visiva immediata dello stato operativo del sistema verso l'esterno. Il suo comportamento è strettamente legato alle transizioni di stato del servizio centrale: deve attivare la modalità lampeggiante per indicare che il sistema si trova in uno stato di impegno (engaged), mantenendo questo comportamento durante tutte le fasi di trasferimento dei container.

Dopo aver parlato con il committente si è stabilito che il led è quello presente sul Pico W. Ciò rende necessario modellare il componente come [attore](#), in quanto si troverà sullo stesso nodo e contesto del sonar e dovrà accedere alla rete per comunicare con il resto dell'applicazione.

System pico /* * MODELLO DELL'ANALISTA DEI REQUISITI */ // ----definizione dei messaggi---- // ----definizione del contesto---- Context ctxpico ip [host="localhost" port=8170] // ----definizione degli attori---- QActor sonar context ctxpico { State s0 initial { println("Name starts") color green } } QActor led context ctxpico { State s0 initial { println("Name starts") color yellow } }

Cargoservice

Rappresenta il software centrale responsabile della logica di business e del coordinamento del sistema ed è modellato come [attore](#). Riceve le richieste dalla IOPort, consulta lo stato della stiva per accettare la richiesta (riservando uno slot libero) o rifiutare il carico, gestisce il timeout di presenza del container (30 secondi) e comanda il comportamento del robot e del led di segnalazione.

Formalizzazione dei messaggi:

System cargoservice /* * MODELLO DELL'ANALISTA DEI REQUISITI */ // ----definizione dei messaggi---- Request loadrequest : loadrequest(IOPORT_STATE) Reply retrylater : retrylater(CAUSE_HOLD_STATE) for loadrequest Reply reject : reject(HOLD_STATE) for loadrequest Reply engaged : engaged(RESERVED_SLOT_HOLD_STATE) for loadrequest // ----definizione del contesto---- Context ctxcargoservice ip [host="localhost" port=8120] // ----definizione degli attori---- QActor cargoservice context ctxcargoservice{ [# var HOLD_FULL = false #] State s0 initial{ println("Name starts") color magenta } Transition t0 whenRequest loadrequest -> handleLoadRequest State handleLoadRequest{ onMsg(loadrequest : loadrequest(IOPORT_STATE)) { [# var IoPortState = payloadArg(0).toBoolean() if (IoPortState) { var Cause = "porta_occupata" replyTo loadrequest with retrylater : retrylater(\$Cause, \$HOLD_FULL) println("Name retrylater(\$Cause, \$HOLD_FULL)") color magenta [# } else if (HOLD_FULL) { #] replyTo loadrequest with reject : reject(HOLD_FULL) println("Name reject(HOLD_FULL)") color magenta [# } else { HOLD_FULL = true var Slot = "slot1" #] replyTo loadrequest with engaged : engaged(\$Slot, \$HOLD_FULL) println("Name engaged(\$Slot, \$HOLD_FULL)") color magenta [# #] } } Transition t0 whenRequest loadrequest -> handleLoadRequest } } QActor ioport context ctxcargoservice{ [# var IoPortState = false #] State s0 initial{ println("Name starts") color blue } Transition t0 whenTime 3000 -> pushButtonLibera State pushButtonLibera{ [# IoPortState = false #] println("Name invio loadrequest a cargoservice, ioport libera") color blue request cargoservice -m loadrequest : loadrequest(\$IoPortState) } Transition t1 whenReply engaged -> handleEngaged whenReply reject -> handleReject whenReply retrylater -> handleRetryLater State pushButtonOccupata{ [# IoPortState = true #] println("Name invio loadrequest a cargoservice, ioport occupata") color blue request cargoservice -m loadrequest : loadrequest(\$IoPortState) } Transition t2 whenReply engaged -> handleEngaged whenReply reject -> handleReject whenReply retrylater -> handleRetryLater State handleEngaged{ printCurrentMessage color blue } Transition t3 whenTime 3000 -> pushButtonOccupata State handleReject{ printCurrentMessage color blue } Transition t4 whenTime 3000 -> pushButtonOccupata State handleRetryLater{ printCurrentMessage color blue } Transition t5 whenTime 3000 -> pushButtonLibera } }
--

Stati del sistema cargoservice

Dall'analisi dei requisiti emergono i seguenti stati operativi del servizio:

STATO	DESCRIZIONE	STATO LED
idle	Il sistema è pronto a ricevere nuove richieste di carico	Spento
engaged	Richiesta valida e accettata. In attesa del container (timeout 30 secondi)	Lampeggiante
disengaged	Timeout scaduto senza rilevamento del carico. Ritorno a idle	Spento
service working	Funzionamento atteso	Spento/Lampeggiante
out of service	Rilevata anomalia persistente sul sonar. Sistema bloccato	Rosso

Struttura del sistema

Il sistema cargoservice si presenta come un'architettura eterogenea e distribuita. Ciascun nodo logico identificato dai requisiti opera infatti in un contesto differente: il servizio centrale, il firmware a basso livello per la gestione dei sensori hardware e il simulatore del robot.

In uno scenario caratterizzato da questa forte eterogeneità, basare l'intera infrastruttura di comunicazione esclusivamente su uno scambio di oggetti nativi Java rappresenterebbe un errore di progettazione. Tale approccio introdurrebbe un accoppiamento tecnologico stringente, obbligando ogni singolo nodo a condividere e compilare le stesse classi Java. Questo vincolo ridurrebbe l'estensibilità del sistema e la possibilità di aggiornare i singoli componenti in modo indipendente.

Per superare questo abstraction gap comunicativo, si adotta il metamodello [qak](#) (Quantitative Actor Language). Qak è un Domain-Specific Language (DSL) focalizzato sulla progettazione di sistemi distribuiti ed eterogenei basato sul modello computazionale ad attori. Se si dovesse modellare un sistema ad attori direttamente in Java lo sviluppatore dovrebbe scrivere molto codice infrastrutturale. Con il DSL qak i concetti del dominio vengono espressi come costrutti sintattici di primo livello. qak permette di definire le interazioni tra entità a un alto livello di astrazione, formalizzando i canali di comunicazione attraverso costrutti semantici precisi e disaccoppiati dal linguaggio di programmazione finale:

- Request/Reply**: per interazioni sincrone o transazionali (es. la richiesta di carico del container).
- Dispatch**: per invio asincrono di comandi diretti (es. i controlli inviati ai dispositivi periferici).
- Event**: per l'acquisizione di informazioni generate da sorgenti insolate verso chiunque sia in ascolto (es. i dati telemetrici del sonar).

L'architettura logica che emerge dai requisiti prevede la scomposizione in due macro-nodi computazionali:

- Nodo Applicativo (cargoservice e cargorobot): il fulcro della logica di business.
- Nodo Pico (sonar e led): l'unità per l'elaborazione dei flussi informativi provenienti dal modulo hardware (Pico W).

ATTORI vs POJO

La scelta del paradigma computazionale da associare a ciascun componente del sistema è guidata dalla necessità di gestire flussi informativi concorrenti e asincroni.

Il sistema deve essere in grado di elaborare simultaneamente stimoli esterni multipli e imprevedibili: l'utente che preme il pulsante di carico, il sonar che rileva una variazione di distanza o un'anomalia hardware, e le notifiche di fine marcatrice trasmesse dallo slot5. Per evitare condizioni di stallo (deadlock) e garantire la massima reattività senza ricorrere a complesse logiche di sincronizzazione multi-threading manuale, si opta per il Modello ad Attori per tutti i componenti attivi. Un **attore** è un'entità autonoma dotata di una coda di messaggi privata (mailbox) che elabora gli eventi in modo sequenziale e non bloccante.

Al contrario, le strutture dati puramente passive, che non possiedono un comportamento autonomo ma servono solo a incapsulare lo stato del sistema, vengono modellate come normali oggetti Java (**pojo**).

La tabella seguente riassume la classificazione e la motivazione di ciascun componente:

Entità Logica	Modello Computazionale	Motivazione Progettuale
cargoservice	Attore qak	Rappresenta la macchina a stati centrale dell'applicazione. Deve reagire in modo asincrono agli eventi dei sensori e coordinare le risposte per l'interfaccia utente.
cargorobot	Attore qak	Entità dotata di ciclo di vita autonomo. Riceve le destinazioni di alto livello, monitora lo stato dello spostamento e notifica il completamento dell'azione.
sonar	Attore qak	Agisce come un convertitore di protocollo (Adapter). Riceve le stringhe grezze dal microcontrollore del sonar e le traduce in eventi strutturati per il sistema.
led	Attore qak	Entità passiva di output situata sul nodo Pico: necessita quindi dell'accesso alla rete per poter comunicare con il resto del sistema, motivo per cui viene modellato come attore.
ioport	Attore qak	Disaccoppia la Web GUI dal resto dell'architettura. Smista i comandi del pulsante fisico verso il core e riceve le notifiche per l'aggiornamento del display.
marker	Attore qak	Dispositivo attivo con ciclo di vita autonomo. Reagisce all'arrivo del carico nello slot 5, gestisce la durata della marcatura e notifica in modo asincrono il completamento al cargorobot (markingDone), evitando di bloccare l'orchestrazione principale
hold	POJO (Plain Old Java Object)	Struttura dati passiva che memorizza la mappa della stiva e lo stato degli slot (libero, prenotato, occupato). Viene manipolata direttamente dall'attore cargoservice.

TEST PLAN

Il piano di test per questa fase iniziale di Sprint 0 ha l'obiettivo di verificare formalmente la correttezza logica dei componenti isolabili del sistema prima di procedere all'attivazione dell'architettura distribuita.

Nello specifico, l'implementazione è focalizzata sulla convalida della struttura dati Hold, in quanto rappresenta il nucleo informativo su cui si basano le decisioni di accettazione o rifiuto delle richieste di carico da parte dell'orchestratore.

- Verifica di allocazione iniziale e sequenziale: Accertarsi che a stiva vuota il sistema assegni lo slot 1 e che chiamate successive riservino gli slot in rigido ordine sequenziale (da 1 a 4).
- Gestione della saturazione: Verificare che il metodo isFull() commuti correttamente su true una volta che tutti e 4 gli slot principali si trovano nello stato RESERVED o OCCUPIED.
- Protezione da sovrallocazione: Garantire che ogni tentativo di prenotazione fallisca (ritornando il valore identificativo -1 / NO_SLOT_AVAILABLE) nel momento in cui la stiva ha esaurito la propria capacità.
- Transizione di stato degli slot: Verificare la corretta evoluzione del ciclo di vita di ciascun alloggiamento (da FREE a RESERVED, fino a OCCUPIED o al ripristino a FREE in caso di rilascio/timeout).

Di seguito viene riportata la suite di test funzionali per il componente Hold:

public class HoldTest { private Hold hold; @Before public void init() { hold = new Hold(); } @Test public void testSequentialReservation() { assertEquals(1, hold.reserveNextFreeSlot()); assertEquals(2, hold.reserveNextFreeSlot()); assertEquals(hold.SlotState.RESERVED, hold.getSlotState(1)); assertEquals(hold.SlotState.RESERVED, hold.getSlotState(2)); } @Test public void testIsFullCondition() { assertFalse(hold.isFull()); for (int i = 0; i < Hold.NUM_MAIN_SLOTS; i++) { hold.reserveNextFreeSlot(); } assertTrue(hold.isFull()); } @Test public void testReservationFailureWhenFull() { for (int i = 0; i < Hold.NUM_MAIN_SLOTS; i++) { hold.reserveNextFreeSlot(); } assertEquals(Hold.NO_SLOT_AVAILABLE, hold.reserveNextFreeSlot()); } @Test public void testSlotStateTransition() { int reservedSlot = hold.reserveNextFreeSlot(); assertEquals(1, reservedSlot); assertEquals(hold.SlotState.RESERVED, hold.getSlotState(1)); hold.setSlotOccupied(reservedSlot); assertEquals(Hold.SlotState.OCCUPIED, hold.getSlotState(1)); } @Test public void testReleaseSlot() { int reservedSlot = hold.reserveNextFreeSlot(); hold.releaseSlot(reservedSlot); assertEquals(hold.SlotState.FREE, hold.getSlotState(reservedSlot)); assertFalse(hold.isFull()); } }
--

Oltre ai test automatizzati sul componente Hold, che verificano la correttezza logica interna della struttura dati, si ritiene utile affiancare al piano di test una serie di user story che permettano di descrivere il sistema dal punto di vista dell'utente finale (il cliente che utilizza lo IOPort), mettendo in luce i comportamenti attesi che il livello di esperienza d'uso e non solo di correttezza funzionale.

Il test automatizzato viene progettato per validare i seguenti scenari operativi derivati direttamente dalle specifiche dei requisiti:

- Richiesta di carico accettabile**: Come cliente, voglio premere il pulsante dello IOPort per richiedere il carico di un container, così da ricevere l'assegnazione di uno slot libero se disponibile.
Criterio di accettazione: se la stiva ha almeno uno slot libero e il sistema non è occupato, la richiesta viene accettata e viene restituito il nome dello slot riservato; il LED inizia a lampeggiare.

- Richiesta accettazione a stiva piena**: Come cliente, voglio essere avvisato quando tutti gli slot sono occupati, così da non aspettare inutilmente una risposta positiva.
Criterio di accettazione: se slot1-4 sono tutti occupati, la richiesta riceve risposta reject.

- Sistema occupato o fuori servizio**: Come cliente, voglio ricevere l'indicazione "riprova più tardi" se lo IOPort è già occupato da un altro container o il sistema è fuori servizio, così da capire che devo aspettare.
Criterio di accettazione: se il sistema è engaged/service working oppure in stato out of service, la risposta è retrylater.

- Timeout di conferma carico**: Come cliente che ha ricevuto uno slot assegnato, voglio avere un tempo ragionevole (30 secondi) per posizionare il container nell'area sensore, così da non perdere la prenotazione per un ritardo minimo, ma capire chiaramente se il tempo è scaduto.
Criterio di accettazione: se il container non viene rilevato entro 30 secondi, il sistema torna in stato disengaged e poi idle, liberando lo slot riservato.

- Visualizzazione dello stato del sistema**: Come cliente, voglio vedere sul display lo stato corrente della stiva e un messaggio chiaro sullo stato del servizio, così da sapere se posso effettuare una richiesta.
Criterio di accettazione: il display mostra l'occupazione degli slot 1-4 e uno tra i messaggi "Service working" / "Out of service".

- Segnalazione LED dello stato di impegno**: Come operatore/osservatore del sistema, voglio che il LED lampeggi quando il sistema è impegnato (engaged o service working), così da avere un ritorno visivo immediato senza dover leggere il display.
Criterio di accettazione: il LED lampeggia in engaged e service working, resta spento in idle/disengaged, e diventa rosso/spento in out of service.

- Notifica guasto sonar**: Come manutentore, voglio essere avvisato quando il sonar rileva un'anomalia persistente ($D > DFREE$ per più di 3 secondi), così da poter intervenire prima che il sistema blocchi nuove richieste.
Criterio di accettazione: il sistema passa a out of service e il display mostra il messaggio corrispondente.

DEFINIZIONE DELLO SPRINT1

A conclusione di questa prima fase di analisi (Sprint 0), viene delineato il perimetro d'azione per il prossimo incremento (Sprint 1).

L'obiettivo è concentrarsi sulla logica centrale e sul comportamento del sistema, lasciando a un secondo momento l'integrazione con i dispositivi fisici e i movimenti del robot.

Requisiti Selezionati
Lo Sprint 1 si focalizzerà esclusivamente sul seguente sottoinsieme di specifiche:

- ricevere delle richieste di carico, controllo della stiva tramite la struttura dati interna Hold e generazione delle risposte di accettazione (engaged) o rifiuto (retrylater o reject).
- gestione delle transizioni tra gli stati logici di idle ed engaged, incluso il controllo del timer di 30 secondi che porta allo stato disengaged in caso di mancato arrivo del container.

A emissione dei comandi per accendere/spegnere il lampeggio del LED e dei messaggi di testo per l'aggiornamento dello schermo.

La scelta di questo sottoinsieme permette di testare l'intera logica di controllo del servizio senza dipendere dai movimenti del cargorobot o dai dati del sonar. Verificare che gli stati interni cambino correttamente all'inizio garantisce una base solida prima di passare al sistema distribuito reale.

Goal dello Sprint 1
Sviluppare un prototipo software funzionante incentrato sul componente cargoservice. Il sistema dovrà essere in grado di elaborare stimoli di input fittizi (che simulano la pressione del pulsante dell'IOPort, *button_pushed*), aggiornare coerentemente lo stato di occupazione degli slot della stiva (*hold_status*) e governare i cicli di temporizzazione per il disimpegno del sistema.

Problem analysis

Test plans

Project

Testing

Deployment

Maintenance

Team di Sviluppo

GitHub Project Repository

Arianna Buffoni
arianna.buffoni@studio.unibo.it

Desirée Pellegrini
desiree.pellegrini@studio.unibo.it

Gianluca Varisco
gianluca.varisco@studio.unibo.it